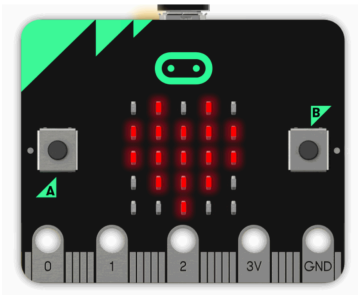
Lab 2: Programming in C and Assembly



Hilary Term 2026

Mark van der Wilk — Lecturer & Adaptations
Mike Spivey — Lecturer & Course Designer

Contents

1. Introduction 1
2. Tasks 2
3. GPIO Connections 2
4. Bonus programs 4

ⓘ **Much to read, not as much to do. Read *before* the scheduled lab session!**

1. Introduction

This lab begins with a program (written entirely in C) that displays a beating heart pattern on the micro:bit's LEDs: it might be an electronic Valentine's card. Your task is to enhance the program so that it shows different patterns when the buttons are pressed.

The `lab2-heart` directory of the lab materials contains the following files, some of them the same as the corresponding files seen before:

File Name	Description
`Makefile`	Build script
`heart.c`	Main program
`hardware.h`	Header file with layout of I/O registers
`startup.c`	Startup code
`nRF51822.ld`	Linker script
`heart-intr.c`	Interrupt-driven static heart program
`blinky.s`	Pure assembly language program for blinking LED

The file `heart.c` contains all the code specific to this program. Using the addresses of hardware registers that are given in the header file `hardware.h`, it configures for output those GPIO pins that are connected to the LED matrix, and for input the two pins that are connected to the buttons. Then it enters a nested loop, where the outer loop (in the main program init) shows two images in alternation, a big heart and a little one, with the little heart shown twice for brief periods in each cycle, giving the impression of a beating heart.

There's another loop (in function show) that looks after the display of each image. On the micro:bit, it's possible to light a single LED by enabling its row and column, or all the LEDs by simultaneously enabling all the rows and all the columns. But to show a specific image, it's necessary to show it one row at a time, multiplexing between the rows fast enough for the flashing to be lost in persistence of vision. To show each row, we activate that row and the columns for the LEDs in that row that should be lit, and then pause for a while before moving on the next row. Things are complicated for the programmer by the fact that, although the LEDs are arranged physically in a 5 x 5 array, they are wired up in a slightly chaotic pattern to make 3 'logical' rows with 9 LEDs in each (and two LEDs missing from one of the rows). Each image is represented in the program by an array of three integers, giving the value that must be set in the I/O register to display each of the rows.

After setting the GPIO lines to display one of the rows of an image, the program enters an innermost loop (in function `delay`) that simply does nothing for a while, until it's time to move on to the next pattern. The delay loop has been written with a carefully chosen number of `nop` instructions (which do nothing but take one cycle) in its body, so that each iteration of the loop takes 8 cycles, or 500ns on a 16MHz machine. The delay in microseconds is doubled before entering the loop.

A delay loop like this works fine in a simple program, but it commits the processor to be doing nothing useful while the delay is counting down. In more complex programs, there will be other work to do, and it will be unacceptable to waste time in a delay loop when the processor could be doing something useful (or even mining Bitcoin!). We will study later the means (interrupts) to allow this, but you are welcome to enhance this program also to use a timer interrupt instead of a delay loop.

2. Tasks

There are various ways you can experiment with this program. For one thing, it's instructive to make the inner loop delay for longer, so that the multiplexing between rows is no longer hidden by persistence of vision.

The main task is to make the program interactive, so that the pattern on the display changes when either button is pressed – from a big heart that flashes to a small heart to a hollow heart that flashes to a filled heart. You will need to make the program sense whether a button is pressed, and determine the bit patterns needed to display the empty heart. Think carefully about the effect you want: should the new patterns appear immediately, or at the beginning of the next heart-beat?

Another possibility is to make patterns on the display fade in and out, by still devoting 5ms to each row in each iteration, but actually illuminating the LEDs (or some of the LEDs) for only part of that time. Each LED is either fully on or fully off, but if it is on for only a fraction of the time, it will appear dimmer.

> ⓘ **Requirements for S**
>
> For an S, you must demonstrate that you understand how to respond to a button press, and you can design a pattern for a multiplexed LED display without using the `IMAGE` macro, and that you can respond to a button press *immediately* rather than after a delay (although there are more, and less elegant ways of doing this).

3. GPIO Connections

In order to design the hollow heart pattern, you'll need to know what each GPIO output bit means. On the V1 micro:bit, there are twelve bits that matter, three to select a row, and nine to select which LEDs in that row are illuminated. The bottom 16 bits of the output register are laid out like this:

```
r3 r2 r1 c9 c8 c7 c6 c5 c4 c3 c2 c1 0 0 0 0
```

Which LEDs each of these GPIO pins are wired to can be found in Figure 1.

The bottom four bits aren't used, but the other twelve bits correspond to the rows and columns. The logical arrangement of LEDs is shown in the diagram above. To show the filled-in heart pattern, we want to light 2.4, 2.5, 3.4, 3.5, 3.6, 3.7, 3.8, 2.2, 1.9, 2.3, 3.9, 2.1, 1.7, 1.6, 1.5, 3.1. To light an LED, we must put a 1 in the right row, and a 0 in the right column, because the cathodes of the LEDs are connected to the column bits. So we get the pattern

```
0 0 1 0 1 0 0 0 1 1 1 1 0 0 0 0 = 0x28f0
0 1 0 1 1 1 1 0 0 0 0 0 0 0 0 0 = 0x5e00
1 0 0 0 0 0 0 0 0 1 1 0 0 0 0 0 = 0x8060
```

and these are the constants embedded in the program.

The good news is that there is actually little need to work out these constants by hand, because the header file hardware.h contains a sneaky macro IMAGE that allows us to write the definition of heart as

```
const unsigned heart[] =
  IMAGE(0,1,0,1,0,
        1,1,1,1,1,
        1,1,1,1,1,
        0,1,1,1,0,
        0,0,1,0,0);
```

The resulting list of expressions is exteremely complicated, but the C compiler is able to reduce each expression to the right single 32-bit constant.

The program already contains code to initialise the pins connected to the two buttons as inputs: they are pins 17 and 26, which `hardware.h` identifies with the symbolic constants `BUTTON_A` and `BUTTON_B`. To test whether each button is pressed, you need to look at the correct bits in the value read from `GPIO_IN`, which can be selected using the masks `BIT(BUTTON_A) = 0x20000` and `BIT(BUTTON_B) = 0x4000000`. As the circuit diagram shows, the buttons are connected between the pin and ground with a pullup resistor. That means the input bit will be 1 when the button is not pressed, and 0 when it is pressed. (The macro `BIT` is also defined in `hardware.h` so that `BIT(x) = (1 << x)`.)

Requirements for S+: Option #1

1. Go through the preprocessor code to determine the list of calculations that is performed to find the 32-bit constant for the bit pattern. Estimate how many assembly operations you need to perform these calculations, for a general input that is not known at compile time.
2. Disassemble the code, and find the assembly instructions that correspond to the `IMAGE` macro. Verify that far fewer instructions are produced by the C compiler than you estimated earlier. Be prepared to demonstrate this to the lab demonstrator.
3. Write a piece of assembly code that calculates an image at *runtime*. The function would need to take in an array with one number (which can be zero or one) per pixel, and output three 32-bit values for controlling the LEDs. Demonstrate this by showing that you can transmit an arbitrary image over the serial port (using the functions from lab0 and lab1), and display this on the LEDs.
4. You may want to think about compressing the image by storing all 25 pixels in a single 32-bit number, but this is optional for the S+.

Requirements for S+: Option #2

Get creative and create a cool little demo with embedded programming skills that you have. This could incorporate more elaborate things to display and richer ways of interacting with the hardware (e.g. detecting a double click).

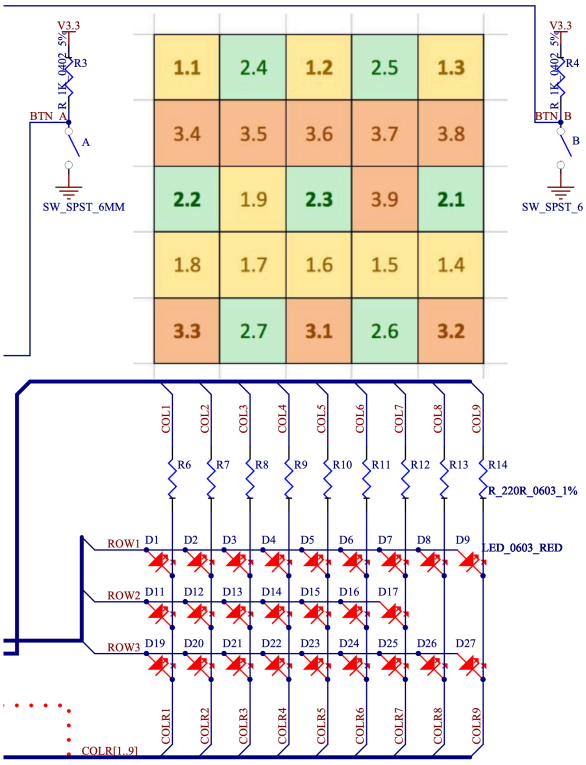


Figure 1: Electrical wiring diagram of LEDs and buttons.

4. Bonus programs

The code in this lab contains a few bonus programs, which will illustrate other parts of the course. You may want to refer to these when concepts appear.

`heart-intr.c`

The program in `heart-intr.c` is interrupt-driven, and displays a static heart pattern without using delay loops. Use

```
make heart-intr.hex
```

to generate a downloadable file. One of the problems on Sheet 3 asks about enhancing a program like this to show a beating heart.

`blinky.s`

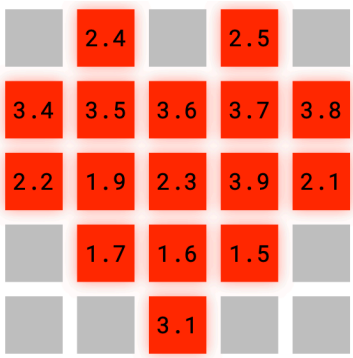


Figure 2: LEDs that need to be switched on to display a heart.

Almost all of the programs in the course rely on the code in `startup.c` to initialise the micro:bit when it comes out of reset. The assembly language file `blinky.s` simplifies this by having the *complete* program in one file of assembly. All it does is blinks one of the LEDs. Use

```
make blinky.hex
```

to generate the downloadable file.

The program establishes values for just the first two elements of the vector table, giving the initial values of the stack pointer and the program counter; since it enables no interrupts, the remaining vectors need not appear. The program contains a subroutine with a delay loop, and a main program that initialises the relevant GPIO pins as outputs, then uses the delay subroutine to flash the central LED.